



CYBERSTER

Cyber Security Internship Program

INTERNSHIP WEEKLY REPORT

Week 3 — Vulnerability Assessment & Initial Exploitation (Red Team Track)

Topic: Advanced Web Directory & Parameter Discovery, Service-to-CVE Mapping & Exploit Research, and CMS / Third-Party Web-Application Vulnerability Scanning — performed against Metasploitable2, an intentionally vulnerable, legally authorized practice target

Student Name: Umar Farooq

Batch#: 2

Instructor: Umar Niaz

Submission Date: July 11, 2026

Contents

Executive Summary	4
Learning Objectives	5
Environment Setup.....	5
Operating System.....	5
Virtual Machine Configuration.....	6
Network Configuration	6
Lab Architecture	6
Installed Tools.....	6
Host Discovery & Connectivity Verification	7
Task 1: Advanced Web Directory & Parameter Discovery.....	9
Objective	9
Theory / Background	9
Tools Used	9
Procedure	9
Commands Used	9
Output / Results.....	10
Screenshots & Evidence	11
Observations	12
Key Learning Points	12
Security Relevance.....	12
Task 2: CVE Mapping & Exploit Research.....	13
Objective	13
Theory / Background	13
Tools Used	13
Procedure	13
Commands Used	13
Output / Results.....	14
Screenshots & Evidence	15
Observations	16
Key Learning Points	16
Security Relevance.....	16

Task 3: CMS & Third-Party Web-Application Vulnerability Scanning	17
Objective	17
Theory / Background	17
Tools Used	17
Procedure	17
Commands Used	18
Output / Results.....	18
Screenshots & Evidence	18
Observations	18
Key Learning Points	19
Security Relevance.....	19
Consolidated Service, Vulnerability & Exploit Table	20
Actionable Insights	20
Technical Concepts Learned.....	21
Challenges Faced	22
Best Practices	22
Weekly Summary Table	23
Conclusion	23
References	24
Appendix.....	24
A. Raw Command Outputs Retained.....	24
B. Screenshot Index	25
C. Submission Checklist	25

Executive Summary

In the third week of my Cyber Security Internship at Cyberster, on the Red Team track, I moved from the pure scanning and enumeration work of Week 2 into the vulnerability-assessment and initial-exploitation phase of an engagement. Where Week 2 answered the question of what is running on the target, Week 3 was built around the harder and more valuable question that follows it: given a specific service and a specific version, what known weakness can actually be used to get in, and where is that weakness most likely to be hiding. I carried out the entire week against the same isolated Metasploitable2 target used in Week 2, reachable only from my Kali attack workstation on a private Host-Only VirtualBox segment, so that every technique in this report was exercised against a machine I am explicitly authorized to test.

The week was organized around three practical tasks that together form the front half of a real intrusion. I began with advanced web content discovery, using ffuf and Gobuster with the SecLists wordlists to brute-force hidden directories and files that are never linked from the site's homepage, then used Arjun to fuzz for hidden GET/POST parameters on a dynamic endpoint. This surfaced a set of high-value entry points — phpMyAdmin, TikiWiki, TWiki, a WebDAV directory, an exposed phpinfo page, and live request parameters — that a casual browse of the site would have missed entirely. I then moved into CVE mapping and exploit research, re-establishing an authoritative version baseline with an Nmap service-and-default-script scan and feeding each confirmed version string into Searchsploit, cross-referencing the results against the NVD and Exploit-DB with a deliberate focus on remote-code-execution and foothold-grade issues rather than denial-of-service noise. The third task addressed CMS and third-party web-application weaknesses: I fingerprinted the web stack with WhatWeb, established that the target's web root is not a WordPress or Joomla install, and pivoted the CMS-scanning discipline onto the genuinely vulnerable third-party applications the host actually serves.

By the end of the week I had a consolidated table linking every exposed service to a specific, named CVE and, where one exists, to a concrete public exploit path on disk; a prioritized view of which single finding I would attempt first for initial access; and a clear, honest account of where the syllabus's default tooling (WPScan, JoomScan) did and did not apply to this particular target. Just as importantly, I built a working understanding of why hidden directories and untidy parameters are where web vulnerabilities actually live, why a version string is worthless until it is

matched against an authoritative vulnerability source, and why an automated exploit search is a starting point for verification rather than a finished finding. Week 3 closes with a repeatable assessment workflow that extends directly from the Week 2 enumeration foundation and sets up the controlled exploitation work to follow.

Learning Objectives

By the end of Week 3, I was expected to be able to:

Perform advanced web content discovery with ffuf, Gobuster and the SecLists wordlists, and explain why non-indexed directories and files (such as /phpMyAdmin, /twiki, /dav and exposed /phpinfo pages) frequently represent the real attack surface of a web application.

Use a dedicated parameter-discovery tool such as Arjun to identify hidden GET and POST parameters on a dynamic endpoint, and articulate why an injectable parameter is often the precise point at which a web vulnerability becomes exploitable.

Re-establish an authoritative service-and-version baseline with an Nmap version and default-script scan, and treat that baseline as the single source of truth for all subsequent vulnerability research.

Map a confirmed service version to a specific, named CVE using the Searchsploit command-line tool, and independently verify each mapping against the National Vulnerability Database and Exploit-DB rather than trusting the tool's own labelling.

Distinguish remote-code-execution and foothold-grade vulnerabilities from lower-value denial-of-service and information-disclosure findings, and prioritize research accordingly.

Fingerprint a web application's technology stack with WhatWeb, determine whether a CMS-specific scanner such as WPScan or JoomScan is actually applicable to the target, and redirect CMS-scanning discipline onto whatever third-party web applications the host genuinely runs.

Judge whether an identified vulnerability is realistically exploitable in context — checking version ranges, preconditions and the existence of a working public exploit — before treating it as a confirmed initial-access candidate.

Produce a documented, evidence-backed vulnerability-assessment report, complete with a consolidated service-to-CVE-to-exploit table, that a supervisor or client could act on directly.

Environment Setup

Operating System

All attacking-side work for Week 3 was carried out from the same Kali Linux (rolling release) installation used in Weeks 1 and 2, accessed through the terminal session shown throughout this report as `mutex@kali`. This week's evidence and output files were kept in the working directory `~/Desktop/Cyberster/week3`, separate from the earlier weeks' artifacts so that the directory-

discovery, exploit-research and fingerprinting output for this week could not be confused with the previous scanning work.

Virtual Machine Configuration

As in Week 2, the target was Rapid7's official Metasploitable2 image, imported into VirtualBox with 512 MB of RAM and a single virtual CPU core and left in its default, intentionally vulnerable state. I booted the target once and logged in with the published msfadmin/msfadmin credentials purely to confirm it had come up correctly and to read its assigned address from ip a before returning to Kali for the remainder of the week's work.

Network Configuration

Both machines shared the same isolated Host-Only VirtualBox segment. The Kali attacker resolved to 10.0.2.15 and the Metasploitable2 target to 10.0.2.3 for the duration of testing, with the target's own web banner again warning that the VM must never be exposed to an untrusted network. This isolation was treated as a mandatory safety control rather than an incidental detail, since the host carries genuine, unauthenticated remote-code-execution vulnerabilities that would be dangerous on any routable network.

Lab Architecture

The architecture was the same simple two-host isolated segment as Week 2: a single Kali Linux attacker issuing every discovery scan, exploit search and fingerprinting probe in this report, and a single Metasploitable2 target responding on a private network with no route to the internet or to the host machine's production LAN. Before beginning the task work I re-confirmed the segment's membership with an ARP-level host-discovery sweep and verified layer-3 reachability to the target with ICMP.

Installed Tools

Tool	Version / Type	Purpose
ffuf	v2.1.0-dev	Fast web directory and file brute-forcing against the FUZZ keyword
Gobuster	v3.8.2	Directory / file enumeration cross-check with status-code filtering
SecLists	Kali package	Discovery wordlists (Web-Content/common.txt, directory-list-2.3-medium)
Arjun	v2.2.7	Hidden HTTP GET/POST parameter discovery
Nmap	7.99, Kali rolling	Service/version baseline and default (-sC) script enumeration
Searchsploit	Exploit-DB CLI	Offline mapping of service versions to public exploit code
WhatWeb	CLI	Web technology-stack and CMS fingerprinting
netdiscover / ip / ping	CLI	ARP host discovery, interface configuration and reachability checks
NVD / Exploit-DB	Web databases	Independent CVE verification and exploit cross-referencing

Host Discovery & Connectivity Verification

Before starting the assessment tasks I re-verified the lab was in a known-good state. An ARP-based host-discovery sweep confirmed exactly two live hosts on the segment, the attacker's own default gateway and the Metasploitable2 target; an ip a on the Kali side confirmed the attacker's 10.0.2.15 address; a five-packet ICMP test to 10.0.2.3 returned zero loss; and a console login on the target confirmed its 10.0.2.3 address directly.

```
(metex@kali) ~/Desktop
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:2b:c3:86 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 459sec preferred_lft 459sec
    inet6 fe80::a00:27ff:fe2b:c386/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 68:9d:37:3b:bd:5b brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

(metex@kali) ~/Desktop
$ ping 10.0.2.3
PING 10.0.2.3 (10.0.2.3) 56(84) bytes of data:
64 bytes from 10.0.2.3: icmp_seq=1 ttl=64 time=0.569 ms
64 bytes from 10.0.2.3: icmp_seq=2 ttl=64 time=0.507 ms
64 bytes from 10.0.2.3: icmp_seq=3 ttl=64 time=0.314 ms
64 bytes from 10.0.2.3: icmp_seq=4 ttl=64 time=0.422 ms
64 bytes from 10.0.2.3: icmp_seq=5 ttl=64 time=0.323 ms
^C
  10.0.2.3 ping statistics ---
  5 packets transmitted, 5 received, 0% packet loss, time 4092ms
 rtt min/avg/max/mdev = 0.314/0.427/0.569/0.100 ms
```

Figure 1: Attacker interface configuration and reachability check

Description: Terminal output of ip a followed by ping 10.0.2.3 on the Kali attacker, confirming the eth0 address 10.0.2.15 and layer-3 reachability to the target.

Observation: All five ICMP echoes returned with 0% packet loss and sub-millisecond round-trip times, confirming a clean, low-latency path to the target before any scanning began.

```
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b9:5a:ff brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.3/24 brd 10.0.2.255 scope global eth0
    inet6 fe80::a00:27ff:feb9:5aff/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$
```

Figure 2: Target IP confirmation on the Metasploitable2 console

Description: VirtualBox console of the Metasploitable2 guest running ip a, showing eth0 bound to 10.0.2.3 on the isolated Host-Only segment.

Observation: Reading the address from the target's own console removed any ambiguity about which host every subsequent command in this report was aimed at.

Session Actions Edit View Help					
Currently scanning: Finished! Screen View: Unique Hosts					
2 Captured ARP Req/Rep packets, from 2 hosts. Total size: 120					
IP	At MAC Address	Count	Len	MAC	Vendor / Hostname
10.0.2.2	08:00:27:02:f2:43	1	60	PCS	Systemtechnik GmbH
10.0.2.3	08:00:27:b9:5a:ff	1	60	PCS	Systemtechnik GmbH

Figure 3: ARP-level host discovery on the lab segment

Description: Host-discovery output showing two unique hosts captured on the 10.0.2.0/24 segment — the gateway at 10.0.2.2 and the target at 10.0.2.3.

Observation: Confirming only the expected hosts were present verified the segment was correctly isolated, with no unintended machines in scope.

Task 1: Advanced Web Directory & Parameter Discovery

Objective

Discover the hidden entry points of the web application running on 10.0.2.3 — the non-indexed directories, files and URL parameters that are not linked from the visible site — on the basis that these unlisted paths and untidy parameters are where exploitable web vulnerabilities most often live.

Theory / Background

A web server almost always exposes far more than the pages a visitor can reach by clicking links. Administrative panels, backup files, development artifacts, configuration pages and third-party applications frequently sit at predictable paths that are simply never referenced from the homepage. Directory brute-forcing closes that gap by requesting thousands of candidate paths from a curated wordlist and recording which ones the server answers for, using the HTTP status code to infer whether a path exists (200), redirects (301/302), is forbidden but present (403), or is absent (404). Parameter discovery works on the same principle one layer deeper: a dynamic page such as `index.php` may quietly accept parameters — an id, a file, a page selector — that never appear in any link but change the application's behaviour when supplied, and those parameters are exactly the injection points where SQL injection, local file inclusion and similar flaws are found. Tools such as Arjun fuzz a known endpoint with a large list of candidate parameter names and detect which ones measurably alter the response.

Tools Used

ffuf v2.1.0-dev and Gobuster v3.8.2 for directory/file brute-forcing against the SecLists wordlists, and Arjun v2.2.7 for hidden-parameter discovery.

Procedure

1. Re-ran the same discovery with Gobuster in directory mode as an independent cross-check, using 404 as the negative status code and confirming the two tools agreed on the discovered paths.
2. Reviewed the combined results for high-value directories — administrative interfaces, third-party applications, development and configuration artifacts.
3. Ran Arjun against the dynamic endpoint `http://10.0.2.3/mutillidae/index.php` to fuzz for hidden GET/POST parameters, letting the tool detect parameters by their effect on HTTP headers and response body length.

Commands Used

```
# Directory / file brute-force with ffuf
ffuf -u http://10.0.2.3/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt

# Independent cross-check with Gobuster
```

```
gobuster dir -u http://10.0.2.3 \  
-w /usr/share/seclists/Discovery/Web-Content/common.txt  
  
# Hidden parameter discovery with Arjun  
arjun -u http://10.0.2.3/mutillidae/index.php
```

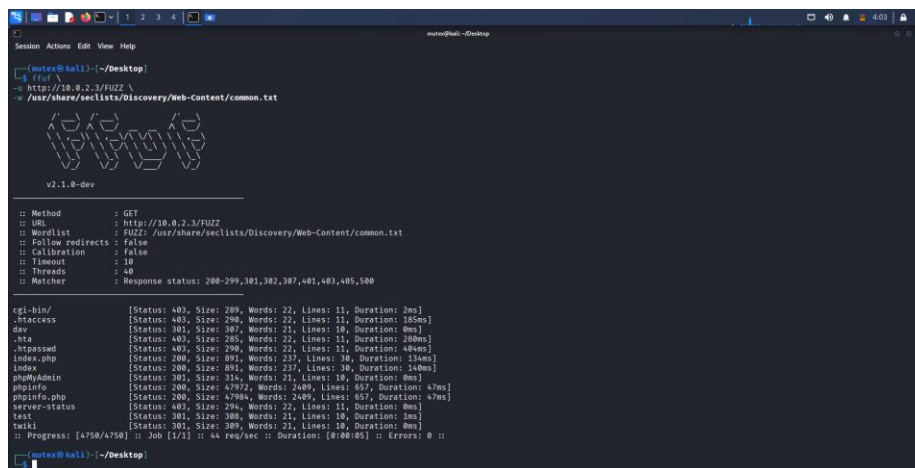
Output / Results

The ffuf run completed 4,750 requests in about five seconds and returned a consistent set of live paths on the target: index and index.php (200), phpinfo and phpinfo.php (200), phpMyAdmin (301), dav (301), doc (301), icons (301), test (301), twiki (301) and tikiwiki (301), together with cgi-bin, .hta, .htaccess, .htpasswd and server-status returning 403 (forbidden but confirmed present). Gobuster, run independently against the same wordlist, reproduced the same set of directories and redirects, which gave me confidence that the discovery was a genuine reflection of the server's layout rather than an artifact of one tool's matching logic.

Several of these paths are immediately significant. The phpMyAdmin directory exposes a database administration interface; the twiki and tikiwiki directories are third-party wiki applications with long histories of documented remote-code-execution vulnerabilities; the dav directory indicates a WebDAV-enabled path that can permit file upload; and phpinfo.php discloses the full PHP configuration, including versions and paths, to any unauthenticated visitor. None of these were linked from the site's homepage.

The Arjun run against the Mutillidae index endpoint probed the target for stability, analysed the HTTP responses for anomalies, and confirmed two live parameters: PHPSESSID, detected from the HTTP headers, and page, detected from a measurable change in response body length. The page parameter in particular is a classic file-/content-selector of exactly the kind that local-file-inclusion and path-traversal testing targets, making it a concrete, named starting point for later injection work.

Screenshots & Evidence



```
Session Actions Edit View Help
--(mutex@kali)~(/Desktop)
$ ffuf -u http://10.0.2.3/FUZZ -w /usr/share/seclists/Discovery/Web-Content/common.txt
  _____
  \         /
  /  \_____/
  \_____/

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.0.2.3/FUZZ
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

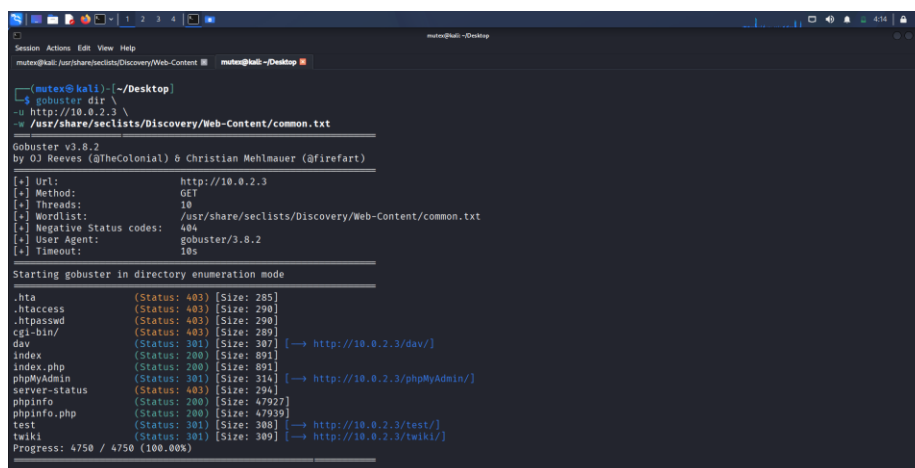
cgi-bin/      [Status: 403, Size: 289, Words: 22, Lines: 11, Duration: 2ms]
.htaccess     [Status: 403, Size: 290, Words: 22, Lines: 11, Duration: 185ms]
dav           [Status: 301, Size: 307, Words: 21, Lines: 10, Duration: 8ms]
.hta          [Status: 403, Size: 285, Words: 22, Lines: 11, Duration: 208ms]
.htpasswd     [Status: 403, Size: 290, Words: 22, Lines: 11, Duration: 40ms]
index.php     [Status: 200, Size: 891, Words: 237, Lines: 30, Duration: 13ms]
index        [Status: 200, Size: 891, Words: 237, Lines: 30, Duration: 140ms]
phpMyAdmin    [Status: 301, Size: 314, Words: 21, Lines: 10, Duration: 8ms]
phpinfo       [Status: 200, Size: 47972, Words: 2409, Lines: 657, Duration: 47ms]
phpinfo.php   [Status: 200, Size: 47984, Words: 2409, Lines: 657, Duration: 47ms]
server-status [Status: 403, Size: 294, Words: 22, Lines: 11, Duration: 8ms]
test         [Status: 403, Size: 294, Words: 22, Lines: 11, Duration: 8ms]
twiki        [Status: 301, Size: 308, Words: 21, Lines: 10, Duration: 1ms]
twiki        [Status: 301, Size: 309, Words: 21, Lines: 10, Duration: 8ms]
:: Progress: [4750/4750] :: Job [1/1] :: 44 req/sec :: Duration: [0:00:05] :: Errors: 0 ::

--(mutex@kali)~(/Desktop)
```

Figure 4: ffuf directory brute-force against the web root

Description: Terminal output of ffuf -u http://10.0.2.3/FUZZ against the SecLists Web-Content wordlist, listing each discovered path with its status code, size, and word/line counts.

Observation: The scan surfaced phpMyAdmin, twiki, tikiwiki, dav and an exposed phpinfo.php, none of which are linked from the homepage — a direct demonstration of hidden attack surface.



```
Session Actions Edit View Help
--(mutex@kali)~(/Desktop)
mutex@kali: /usr/share/seclists/Discovery/Web-Content  mutex@kali ~ (/Desktop)
$ gobuster dir -u http://10.0.2.3 -w /usr/share/seclists/Discovery/Web-Content/common.txt
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[*] Url: http://10.0.2.3
[*] Method: GET
[*] Threads: 10
[*] Wordlist: /usr/share/seclists/Discovery/Web-Content/common.txt
[*] Negative Status codes: 404
[*] User Agent: gobuster/3.8.2
[*] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 285]
.htaccess (Status: 403) [Size: 290]
.htpasswd (Status: 403) [Size: 290]
cgi-bin/ (Status: 403) [Size: 289]
dav (Status: 301) [Size: 307] [→ http://10.0.2.3/dav/]
index (Status: 200) [Size: 891]
index.php (Status: 200) [Size: 891]
phpMyAdmin (Status: 301) [Size: 314] [→ http://10.0.2.3/phpMyAdmin/]
phpinfo (Status: 403) [Size: 294]
phpinfo.php (Status: 200) [Size: 47927]
server-status (Status: 200) [Size: 47939]
test (Status: 301) [Size: 308] [→ http://10.0.2.3/test/]
twiki (Status: 301) [Size: 309] [→ http://10.0.2.3/twiki/]
Progress: 4750 / 4750 (100.00%)
```

Figure 5: Gobuster directory enumeration cross-check

Description: Terminal output of gobuster dir against http://10.0.2.3 with the same wordlist, showing the discovered directories together with their resolved redirect targets.

Observation: Gobuster independently reproduced the ffuf result set, confirming the discovered layout through a second tool with a different matching implementation.

```
(mutex@kali)-[~/Desktop]
$ arjun -u http://10.0.2.3/mutillidae/index.php

  ____
 (  __| | /  (  /) v2.2.7
  /____\

[*] Scanning 0/1: http://10.0.2.3/mutillidae/index.php
[*] Probing the target for stability
[*] Analysing HTTP response for anomalies
[*] Logicforcing the URL endpoint
[v] parameter detected: PHPSESSID, based on: http headers
[v] parameter detected: page, based on: body length
[+] Parameters found: PHPSESSID, page
```

Figure 6: Arjun hidden-parameter discovery on the Mutillidae endpoint

Description: Terminal output of `arjun -u http://10.0.2.3/mutillidae/index.php`, showing the tool probing for stability and then confirming the parameters `PHPSESSID` and `page`.

Observation: The `page` parameter was detected purely from a change in response body length, isolating a live, unlinked input that would be the natural focus of later inclusion or traversal testing.

Observations

The most useful lesson of this task was how much of the target's real functionality was invisible from the browser. Two independent brute-forcers agreeing on a directory list that included a database console and two historically vulnerable wiki applications made concrete the idea that content discovery, not the visible site, defines the true attack surface. Equally, Arjun turning an ordinary-looking `index.php` into a named, testable input point showed that parameter discovery is not an optional extra but the step that converts a page into a candidate vulnerability.

Key Learning Points

I learned to treat directory and parameter discovery as the first real deliverable of a web assessment rather than a warm-up, and to cross-check a wordlist scan with a second tool before trusting its output. I also learned to read status codes actively — a 403 is a confirmed, present path worth noting, not a dead end — and to regard a discovered parameter such as `page` as a specific, documentable lead for the injection testing that follows.

Security Relevance

Unlinked directories and undocumented parameters are among the most common real-world causes of web compromise: an exposed admin panel, a forgotten backup, a WebDAV upload path or an unsanitised file parameter each turns a seemingly static site into an entry point. Enumerating them first is what makes the vulnerability research in the next task targeted rather than speculative.

Task 2: CVE Mapping & Exploit Research

Objective

Take the exact service versions running on 10.0.2.3 and find working, verifiable exploits for them, focusing on remote-code-execution and foothold-grade issues, so that the enumeration data from Week 2 is translated into a concrete, prioritized set of initial-access candidates.

Theory / Background

A vulnerability database indexes almost every entry by product and precise version, which means the version string is the key that unlocks all subsequent research. Searchsploit is the offline command-line interface to the Exploit-DB archive that ships with Kali; querying it with a product and version returns the titles and on-disk paths of any public exploits Exploit-DB holds for that software, along with a marker for Metasploit modules. Crucially, a hit in Searchsploit is a lead, not a conclusion: the exact version range, the required preconditions and the reliability of the exploit all have to be checked against an authoritative source such as the National Vulnerability Database or the Exploit-DB web entry before the finding can be treated as a genuine, exploitable weakness. This task deliberately prioritized issues offering remote code execution or a direct foothold over denial-of-service and pure information-disclosure entries, since the former are what actually advance an engagement.

Tools Used

Nmap 7.99 (service/version detection with default scripts, -sV -sC) to re-establish the version baseline, and Searchsploit against the Exploit-DB archive, cross-referenced with the NVD and Exploit-DB web databases.

Procedure

4. Ran an Nmap service-and-default-script scan (-sV -sC) against 10.0.2.3 to reconfirm the exact version string of every open service and capture the default-script context around each one.
5. For each confirmed version — vsftpd 2.3.4, Samba 3.0.20, ProFTPD 1.3.1, Apache Tomcat, Apache 2.2.8 — queried Searchsploit for matching public exploits.
6. Read each candidate exploit's title and path, separating remote-code-execution and backdoor entries from denial-of-service and information-disclosure noise.
7. Cross-referenced each shortlisted exploit against the NVD and Exploit-DB to confirm the CVE identifier, the affected version range and the existence of a working module before recording it as a verified finding.

Commands Used

```
# Re-establish an authoritative version baseline
nmap -sV -sC 10.0.2.3

# Map each confirmed version to public exploit code
```

```
searchsploit vsftpd 2.3.4
searchsploit samba 3.0.20
searchsploit proftpd 1.3.1
searchsploit tomcat
searchsploit apache 2.2.8
```

Output / Results

The Nmap -sV -sC scan reconfirmed the full service inventory and, through its default scripts, added valuable context: the ftp-anon script confirmed anonymous FTP login is allowed on vsftpd 2.3.4 (FTP code 230); the ssh-hostkey script returned OpenSSH 4.7p1 Debian 8ubuntu1; and further down the port range it identified PostgreSQL DB 8.3.0–8.3.7 on 5432, VNC protocol 3.3 on 5900, an open X11 server on 6000, UnrealIRCd on 6667, Apache Jserv (AJP13) on 8009 and Apache Tomcat/Coyote JSP engine 1.1 (Tomcat 5.5) on 8180. The SMB host scripts again reported Samba 3.0.20-Debian with message signing disabled.

Feeding these versions into Searchsploit produced a clear set of foothold-grade candidates. The vsftpd 2.3.4 query returned vsFTPD 2.3.4 — Backdoor Command Execution as both a standalone script (unix/remote/49757.py) and a Metasploit module (unix/remote/17491.rb), corresponding to CVE-2011-2523. The Samba 3.0.20 query returned Samba 3.0.20 < 3.0.25rc3 — 'Username map script' Command Execution (Metasploit) at unix/remote/16320.rb, corresponding to CVE-2007-2447, an unauthenticated remote command execution. The Tomcat query returned a long list of manager and AJP entries, including the AJP 'Ghostcat' file read/inclusion module (multiple/webapps/48143.py and 49039.rb, CVE-2020-1938), while the Apache 2.2.8 query surfaced the classic mod_ssl 'OpenFuck' remote buffer overflow (764.c) and a cgi-bin remote code execution entry. The ProFTPD 1.3.1 query returned no results, which I recorded honestly rather than forcing a match.

Separately, the UnrealIRCd service on port 6667 is well known to carry the UnrealIRCd 3.2.8.1 backdoor (CVE-2010-2075), and the distccd service historically shipped with Metasploitable2 carries CVE-2004-2687; both were noted as additional, well-documented foothold options to be confirmed against their exact banners. In every case I checked the exploit's stated version range against the version string from the baseline scan rather than against the service family in general.

Screenshots & Evidence

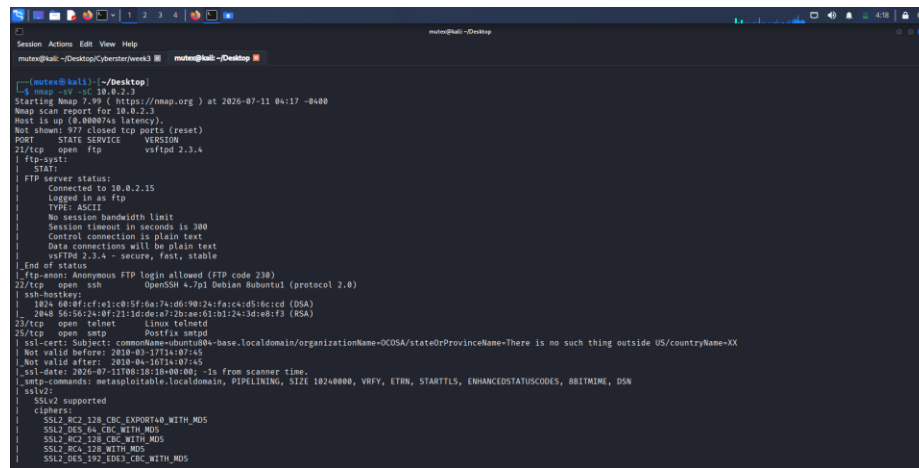


Figure 7: Nmap service and default-script baseline (FTP, SSH, SMTP)

Description: Terminal output of `nmap -sV -sC 10.0.2.3`, showing vsftpd 2.3.4 with the ftp-anon script confirming anonymous login (FTP code 230) and OpenSSH 4.7p1 on port 22.

Observation: The default scripts added exploitation-relevant context to the version strings — anonymous FTP access is itself a standalone finding independent of the version-specific backdoor.

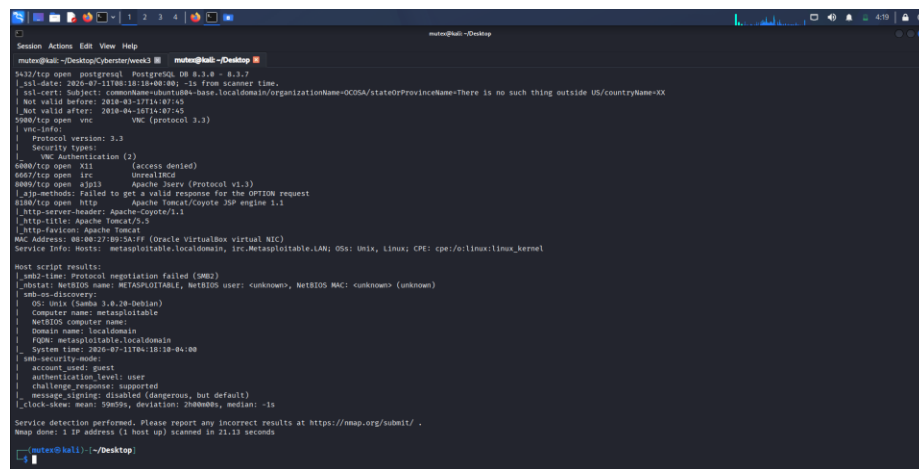


Figure 8: Nmap service baseline (PostgreSQL, VNC, IRC, Tomcat, SMB)

Description: Continuation of the same scan, identifying PostgreSQL 8.3, VNC 3.3, UnrealIRCd, Apache Jserv/AJP13 and Apache Tomcat/Coyote 1.1, plus the Samba 3.0.20-Debian SMB host scripts.

Observation: This lower port range exposed several additional remote-code-execution candidates — UnrealIRCd, Tomcat/AJP and the SMB stack — all of which fed directly into the Searchsploit research.

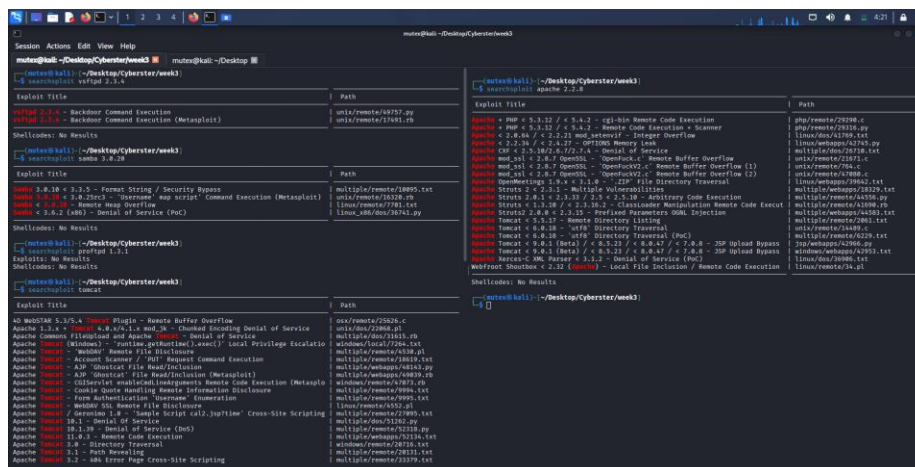


Figure 9: Searchsploit exploit research across the confirmed versions

Description: A multi-pane terminal showing searchsploit queries for vsftpd 2.3.4, samba 3.0.20, proftpd 1.3.1, tomcat and apache 2.2.8, each listing exploit titles and on-disk paths.

Observation: vsFTPD 2.3.4 and Samba 3.0.20 both returned confirmed backdoor / command-execution modules, while ProFTPD 1.3.1 honestly returned no results — a clean illustration of separating real leads from empty ones.

Observations

The clearest takeaway from this task was the difference between a version number and an exploit. Nmap and Searchsploit together turned 'vsftpd 2.3.4 is running' into 'this exact version carries CVE-2011-2523, with a Python script and a Metasploit module already on disk at a known path' — but only because I checked the version range and confirmed the CVE rather than accepting the tool's title at face value. The empty ProFTPD result was as instructive as the positive ones: it showed that not every service yields an exploit, and that recording a negative honestly is part of an accurate assessment.

Key Learning Points

I learned to treat Searchsploit output as a set of hypotheses to verify, not a finished report — checking each candidate's version range and preconditions against the NVD and Exploit-DB, and confirming a working module exists, before calling a finding exploitable. I also learned to prioritize deliberately, filtering the noise of denial-of-service and information-disclosure entries down to the small number of remote-code-execution and backdoor issues that genuinely offer a foothold.

Security Relevance

This task is the exact hinge between assessment and exploitation. Every intrusion that begins with a public exploit begins with this mapping — an unpatched version quietly running in production, matched to a CVE and a ready-made module. Doing it methodically, and verifying before acting, is what separates a credible, prioritized vulnerability report from a raw list of scary-sounding version numbers.

Task 3: CMS & Third-Party Web-Application Vulnerability Scanning

Objective

Identify weaknesses specific to content-management and third-party web applications on the target: fingerprint the web technology stack, determine whether a CMS-specific scanner such as WPScan or JoomScan is applicable, and map the third-party applications the host actually runs to their known vulnerabilities — on the principle that third-party plugins and applications are usually the weakest link.

Theory / Background

Content-management systems and the third-party plugins bolted onto them are a perennial source of compromise, because a site owner may keep the core platform patched while an abandoned plugin or a bundled wiki application quietly rots with a known, exploitable flaw. Specialised scanners exist for the major platforms — WPScan for WordPress, JoomScan for Joomla — and they are extremely effective, but only when the target actually runs that platform. The essential first step is therefore fingerprinting: a tool such as WhatWeb inspects response headers, cookies, meta tags and body content to identify the server, language, framework and any recognised CMS, so that the right specialist scanner can be chosen — or, just as usefully, so that an inapplicable one can be ruled out and effort redirected to whatever third-party applications the host genuinely serves.

Tools Used

WhatWeb for technology-stack and CMS fingerprinting, informed by the directory-discovery results from Task 1, with WPScan and JoomScan considered as the platform-specific scanners for the CMS families they target.

Procedure

8. Ran WhatWeb against `http://10.0.2.3` to fingerprint the server, scripting language and any recognisable CMS at the web root.
9. Compared the fingerprint against the directory-discovery results from Task 1 to determine which recognised third-party applications were actually present on the host.
10. Assessed whether WPScan or JoomScan applied to the target's web root, and — finding no WordPress or Joomla core there — redirected the CMS-scanning discipline onto the genuinely vulnerable third-party applications the host serves (TWiki, TikiWiki, phpMyAdmin).
11. Mapped each of those third-party applications to its documented, well-known vulnerabilities for inclusion in the consolidated table.

Commands Used

```
# Fingerprint the web technology stack / CMS
whatweb http://10.0.2.3

# Platform-specific scanners (applied only where the platform is present)
# wpscan --url http://<target>/<wordpress-path>/ --enumerate vp,u
# joomscan --url http://<target>/<joomla-path>/
```

Output / Results

WhatWeb returned a clean fingerprint of the web root: a 200 OK response served by Apache 2.2.8 on Ubuntu Linux, with WebDAV version 2 enabled and X-Powered-By disclosing PHP 5.2.4-2ubuntu5.10, and a country field of RESERVED/ZZ consistent with the private lab address. Crucially, the fingerprint carried no WordPress or Joomla signature at the web root, which meant that running WPScan or JoomScan against `http://10.0.2.3/` directly would not have been meaningful — an honest, useful negative result rather than a forced scan.

The productive interpretation, combining the fingerprint with the directory discovery from Task 1, is that the target's web-application risk lives in the third-party applications it hosts rather than in a single CMS core. The TWiki application discovered at `/twiki/` is associated with CVE-2005-2877, a command-execution flaw in its debugenableplugins handling. The TikiWiki application at `/tikiwiki/` is associated with CVE-2007-5423, a command-execution issue in its `graph_formula` feature, among other historical flaws. The phpMyAdmin instance at `/phpMyAdmin/` is associated with CVE-2009-1151, a static-code-injection flaw in its setup script. Each of these is precisely the kind of outdated third-party application that the CMS-scanning discipline is designed to catch, and each represents a stronger, more specific web entry point than the generic Apache/PHP stack itself.

Screenshots & Evidence



Figure 10: WhatWeb technology-stack and CMS fingerprint

Description: Terminal output of `whatweb http://10.0.2.3`, reporting Apache 2.2.8 on Ubuntu Linux, WebDAV 2, and PHP 5.2.4-2ubuntu5.10 via the X-Powered-By header.

Observation: The fingerprint showed no WordPress or Joomla core at the web root, ruling WPScan/JoomScan out for the root and redirecting the CMS discipline onto the host's TWiki, TikiWiki and phpMyAdmin applications.

Observations

The honest lesson of this task was that choosing the right tool depends entirely on fingerprinting first. It would have been easy to run WPScan against the target simply because the syllabus names it, and to present an empty or irrelevant result as if it were a finding. WhatWeb instead told me plainly that the web root is a plain Apache/PHP stack with no WordPress or Joomla core, which

let me redirect the CMS-scanning discipline onto the third-party wiki and database applications that the host genuinely runs and that genuinely carry known CVEs.

Key Learning Points

I learned that CMS scanning begins with fingerprinting, not with a scanner, and that ruling a tool out is itself a legitimate and valuable result. I also learned, concretely, why third-party applications are described as the weakest link: the target's most actionable web weaknesses were not in its core server at all but in the ageing TWiki, TikiWiki and phpMyAdmin applications bolted on beside it, each mapping to a specific, documented command-execution or code-injection CVE.

Security Relevance

In real engagements the CMS and its plugins are frequently the softest way in, and the same discipline applies whether the platform is WordPress, Joomla or a bundled wiki: fingerprint accurately, choose the applicable scanner, and pay special attention to outdated third-party components. The value of this task was learning to apply that discipline correctly even when the headline tools named in the brief did not fit the target in front of me.

Consolidated Service, Vulnerability & Exploit Table

The table below consolidates the version baseline from Task 2 with the CVE mappings and public exploit paths confirmed through Searchsploit and cross-referenced against the NVD and Exploit-DB, together with the web-application findings from Tasks 1 and 3, into the single assessment table required by this week's deliverable.

Service / Port	Version	CVE / Weakness	Exploit Path / Note
FTP / 21	vsftpd 2.3.4	CVE-2011-2523 — backdoor command execution (unauth); anonymous login also enabled	unix/remote/49757.py; Metasploit 17491.rb
SSH / 22	OpenSSH 4.7p1	Debian OpenSSL predictable-PRNG era build; weak legacy ciphers	Credential attacks; no direct RCE
Telnet / 23	Linux telnetd	Cleartext credentials and session data in transit	Traffic capture / credential theft
SMTP / 25	Postfix smtpd	User enumeration via VRFY/EXPN	Requires further verification
HTTP / 80	Apache 2.2.8 / PHP 5.2.4	mod_ssl 'OpenFuck' overflow; cgi-bin RCE; hosts vulnerable third-party apps	unix/remote/764.c; see web-app rows
Web app	TWiki (/twiki/)	CVE-2005-2877 — debugenableplugins command execution	Third-party app; strong web entry point
Web app	TikiWiki (/tikiwiki/)	CVE-2007-5423 — graph_formula command execution	Third-party app; strong web entry point
Web app	phpMyAdmin (/phpMyAdmin/)	CVE-2009-1151 — setup.php static code injection	Exposed DB admin interface
Web param	Mutillidae index.php	Hidden 'page' parameter — LFI / traversal candidate	Discovered via Arjun
SMB / 139,445	Samba 3.0.20-Debian	CVE-2007-2447 — 'username map script' RCE (unauth)	unix/remote/16320.rb (Metasploit)
IRC / 6667	UnrealIRCd	CVE-2010-2075 — 3.2.8.1 backdoor command execution	linux/remote/16922.rb (Metasploit)
AJP / 8009	Apache Jserv (AJP13)	CVE-2020-1938 — 'Ghostcat' file read/inclusion	multiple/webapps/48143.py , 49039.rb
HTTP / 8180	Apache Tomcat 5.5	Manager weak-credential deploy → WAR RCE	Metasploit tomcat_mgr modules
PostgreSQL / 5432	PostgreSQL 8.3.x	Authenticated command execution via UDF	Requires valid/default creds
VNC / 5900	VNC protocol 3.3	Weak/known VNC password exposure	Credential attack

Actionable Insights

Weighing every finding from this week's assessment against the others, port 21 running vsftpd 2.3.4 again stands out as the single most likely initial-access point on the target. The version carries a confirmed, publicly documented backdoor (CVE-2011-2523) that requires no authentication and yields direct root-level command execution once triggered, and it has a ready-made script and Metasploit module already on disk; Task 2 also confirmed anonymous FTP login is independently enabled, giving two separate unauthenticated paths into the same port. The Samba 'username map

script' vulnerability on port 445 (CVE-2007-2447) is a strong second candidate, since the exact detected version — Samba 3.0.20-Debian — falls squarely within the documented vulnerable range and offers unauthenticated remote code execution. On the web side, the ageing TWiki and TikiWiki applications discovered in Task 1 are the most promising avenues, each mapping to a specific command-execution CVE, with the exposed phpMyAdmin console and the Arjun-discovered page parameter as supporting leads. In a real engagement I would validate and, with proper authorization, exploit the vsftpd backdoor first for its low complexity and immediate root shell, holding the Samba and web-application vectors in reserve as reliable alternatives.

Technical Concepts Learned

Web Content Discovery

Brute-forcing a server's hidden directories and files from a wordlist and reading HTTP status codes to infer which paths exist, redirect or are forbidden-but-present. This is what defined the true attack surface in Task 1, well beyond the visible site.

Hidden Parameter Discovery

Fuzzing a dynamic endpoint with candidate parameter names and detecting which ones measurably change the response, isolating unlinked inputs such as the page parameter that are the natural targets for inclusion and injection testing.

Service-to-CVE Mapping

Using a precise version string as the index key into a vulnerability database, and treating an automated exploit-search hit as a hypothesis to verify against the NVD and Exploit-DB rather than a finished finding.

Foothold vs. Noise Prioritisation

Separating remote-code-execution and backdoor entries — the issues that actually advance an engagement — from the denial-of-service and information-disclosure results that dominate raw exploit-search output.

Exploit Verification

Checking an exploit's stated version range, preconditions and the existence of a working public module before treating a vulnerability as realistically exploitable in context.

CMS Fingerprinting

Identifying a web application's server, language and any CMS from its headers and body so that the correct platform-specific scanner can be chosen — or an inapplicable one deliberately ruled out.

Third-Party Application Risk

Recognising that outdated bundled applications and plugins, not the core platform, are frequently the weakest link, as demonstrated by the target's TWiki, TikiWiki and phpMyAdmin installs.

Challenges Faced

The table below documents the genuine issues encountered during the week's practical work, their root causes, how each was resolved, and the lesson taken from it.

Issue	Root Cause	Resolution	Lesson Learned
Directory brute-force returned many 403 responses that were easy to dismiss as dead ends.	Forbidden paths still confirm a resource exists; a 403 is presence plus access control, not absence.	Matched a broad status-code set in ffuf and recorded 403 paths as confirmed-present rather than filtering them out.	Read status codes actively — 403 and 301 are findings, not failures.
The WPScan/JoomScan tools named in the brief did not fit the target's web root.	Metasploitable2's web root is a plain Apache/PHP stack, not a WordPress or Joomla core.	Fingerprinted with WhatWeb first, ruled the CMS scanners out honestly, and redirected the discipline onto the host's TWiki/TikiWiki/phpMyAdmin apps.	Fingerprint before scanning; ruling a tool out is a valid result, not a gap.
Searchsploit output mixed high-value RCE entries with large volumes of DoS and info-disclosure noise.	Exploit-DB indexes every public entry regardless of impact, so a query returns everything for a product.	Filtered results down to remote-code-execution and backdoor entries and verified each CVE and version range against the NVD before recording it.	An exploit-search hit is a lead to verify and prioritise, not a finished finding.
ProFTPD 1.3.1 returned no Searchsploit results despite being an old service.	Not every legacy version has a public, indexed exploit in Exploit-DB.	Recorded the empty result honestly rather than forcing an unrelated match onto the version.	A clean negative is part of an accurate assessment and belongs in the report.

Best Practices

- Only assess hosts you are explicitly authorized to test, and treat an isolated Host-Only lab as a mandatory safety control, especially against a target that carries genuine unauthenticated remote-code-execution flaws such as Metasploitable2.
- Run web content discovery early and cross-check it with a second tool, reading status codes actively so that forbidden-but-present (403) and redirecting (301) paths are captured as real findings rather than discarded.
- Follow directory discovery with dedicated parameter discovery on dynamic endpoints, since an unlinked injectable parameter is often the precise point at which a web vulnerability becomes exploitable.
- Re-establish an authoritative version baseline with an Nmap -sV -sC scan before any CVE research, and treat that baseline as the single source of truth for every subsequent mapping.
- Treat every Searchsploit hit as a hypothesis: verify the CVE, the affected version range and the existence of a working module against the NVD and Exploit-DB before recording the finding, and prioritise remote-code-execution and foothold issues over denial-of-service noise.

- Fingerprint a web application before choosing a CMS scanner, and be willing to rule WPScan or JoomScan out where the platform is absent, redirecting effort onto the third-party applications the host actually runs.
- Document negative and inconclusive results — an empty exploit search, an inapplicable scanner — with the same rigour as positive findings, since an honest limitation is more useful to a reader than an inflated claim.
- Keep a contemporaneous, timestamped record of every command and output file, so that an accurate, evidence-backed report can be assembled from artifacts rather than reconstructed from memory.

Weekly Summary Table

Task	Description	Status
1. Advanced Web Directory & Parameter Discovery	ffuf + Gobuster directory brute-force against SecLists; Arjun hidden-parameter discovery on Mutillidae	Completed
2. CVE Mapping & Exploit Research	Nmap -sV -sC version baseline; Searchsploit mapping of vsftpd, Samba, Tomcat, Apache; NVD/Exploit-DB verification	Completed
3. CMS & Third-Party Web-App Scanning	WhatWeb fingerprinting; WPScan/JoomScan applicability assessment; TWiki/TikiWiki/phpMyAdmin CVE mapping	Completed

Conclusion

Week 3 of my Cyberster internship took me from the scanning and enumeration of Week 2 into the vulnerability-assessment and initial-exploitation phase of a red-team engagement, working entirely against the same isolated, intentionally vulnerable Metasploitable2 target. Starting from the service inventory built in Week 2, I discovered the target's hidden web attack surface, translated its exact service versions into specific, verified CVEs with public exploit paths, and applied CMS-scanning discipline correctly by fingerprinting first and redirecting it onto the third-party applications the host genuinely runs.

The most valuable lesson of the week was the same pattern of verification over trust that ran through Week 2, now applied to exploitation: cross-checking two directory brute-forcers against each other, confirming every Searchsploit hit's CVE and version range against the NVD and Exploit-DB before recording it, and using WhatWeb to rule an inapplicable scanner out rather than forcing an empty result. I also learned, concretely rather than abstractly, that web vulnerabilities live in the unlinked and the outdated — the forgotten directory, the untidy parameter, the ageing bundled application — and that a version number is only a lead until it is matched, verified and prioritised.

Technically, I am now comfortable running and cross-checking web content discovery with ffuf and Gobuster, discovering hidden parameters with Arjun, establishing a version baseline with Nmap default scripts, mapping versions to CVEs and public exploits with Searchsploit and

verifying them independently, and fingerprinting a web stack with WhatWeb to drive the right CMS-scanning decision. Going into the next week, I feel ready to move from this prioritized set of verified, foothold-grade vulnerabilities into the controlled exploitation that turns an identified weakness into an actual foothold on the target — beginning, in all likelihood, with the vsftpd 2.3.4 backdoor identified as this week's top initial-access candidate.

References

- ffuf — Fast web fuzzer, official documentation (github.com/ffuf/ffuf).
- Gobuster — Directory/file, DNS and vhost busting tool, official documentation.
- Daniel Miessler et al. — SecLists, the security tester's companion wordlist collection.
- Arjun — HTTP parameter discovery suite, official documentation.
- Nmap Project — Official Nmap Reference Guide and Nmap Scripting Engine documentation (nmap.org/book).
- Offensive Security — Exploit-DB and the Searchsploit command-line utility (exploit-db.com).
- National Vulnerability Database (NVD) — nvd.nist.gov, used for CVE verification throughout Task 2.
- WhatWeb — Next-generation web scanner and technology fingerprinter, official documentation.
- WPScan and JoomScan — CMS-specific vulnerability scanners, considered for applicability in Task 3.
- OWASP Foundation — Web Security Testing Guide (WSTG), Information Gathering and Configuration Management chapters.
- NIST Special Publication 800-115 — Technical Guide to Information Security Testing and Assessment.
- MITRE ATT&CK — Active Scanning (T1595) and Exploit Public-Facing Application (T1190).
- Rapid7 — Metasploitable2 official documentation and associated Metasploit modules.

Appendix

A. Raw Command Outputs Retained

The following raw output files generated during Week 3 are retained alongside this report and were the direct source for every figure and statistic cited above: task01_ffuf.txt, task01_gobuster.txt, task01_arjun.txt, task02_nmap_sv_sc.txt, task02_searchsploit_vsftpd.txt,

task02_searchsploit_samba.txt, task02_searchsploit_tomcat.txt, task02_searchsploit_apache.txt, and task03_whatweb.txt.

B. Screenshot Index

Figures 1 through 10 in the body of this report correspond one-to-one with the dated screenshot files captured during the week's work, named following the Screenshot_2026-07-11_HH_MM_SS.png convention in the project archive, in the same order they were taken and referenced.

C. Submission Checklist

- Cover Page — included
- Table of Contents — included (auto-generated field; update in Word before submitting)
- Executive Summary — included
- Learning Objectives — included
- Environment Setup — included, with tools table and host-discovery evidence
- Task Documentation (3 tasks) — included, with objective/theory/tools/procedure/commands/output/screenshots/observations/key learnings/security relevance for each
- Commands — included in shaded code blocks for every task
- Screenshots with Captions — included (10 figures with description and observation)
- Consolidated Service, Vulnerability & Exploit Table — included
- Actionable Insights — included
- Technical Concepts Learned — included
- Challenges Faced — included
- Best Practices — included
- Weekly Summary Table — included
- References — included
- Grammar Check — recommended before final submission
- Export as PDF — recommended alongside the .docx, per the Cyberster naming convention Week03_UmarFarooq_Report.pdf